

LEVEL 30

Chapter 28: Mock Interview Program

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 28: Mock Interview Program (20 Mocks)

← [Chapter 27: Question Bank](#) · [Index](#) · Next → [Chapter 29: Evaluation Rubric & Final Framework](#)

How to run these solo: read only the **Candidate Brief**, set a 45-minute timer, and run the full [10-step framework](#) out loud (record yourself if possible — hearing yourself back is uncomfortable and extremely effective). Do not read the Interviewer Notes until after you've finished. Then self-score using [Chapter 29's rubric](#).

How to run these with a partner: whoever plays interviewer reads the full entry, including the notes, and uses the hints/follow-ups exactly as a real interviewer would — offering hints progressively, only as the candidate visibly struggles, not upfront.

Deliberately not included: a full worked solution for each mock. Chapters 16–18 already gave you 30 fully worked problems to learn *from*; this chapter is for testing yourself *cold*, which only works if the answer isn't sitting right below the question.

| Beginner Tier (Mocks 1-5)

Mock 1: Parking Lot System

Candidate Brief: "Design a parking lot management system. It should support multiple vehicle types, track available spots, and handle entry/exit with fee calculation."

Interviewer Notes:

- *Expected areas:* clarify vehicle types and spot types (motorcycle/car/bus, small/medium/large spots), entity/class design, spot-assignment algorithm, fee calculation strategy, concurrency (two cars entering simultaneously, one spot left).
- *Progressive hints if stuck:* (1) "What data would you need to track for each spot?" (2) "What happens if two cars arrive for the last compatible spot at the same instant?" (3) "How would fee calculation differ for a motorcycle vs. a bus?"
- *Follow-ups:* "How would you extend this to multiple parking lot locations?" / "How would you handle a reserved-spot feature for monthly subscribers?"
- *Rubric focus:* Data model clarity, atomicity of spot assignment, edge-case handling (full lot, invalid exit).

Mock 2: URL Shortener with Custom Aliases and Click Analytics

Candidate Brief: "Design a URL shortening service. Users can optionally choose a custom alias, and we need a dashboard showing click counts over time per link."

Interviewer Notes:

- *Expected areas:* [Ch.16, Problem 1](#)'s core, plus: custom alias uniqueness handling, analytics ingestion path kept off the redirect's critical path, time-bucketed click aggregation for the dashboard.
- *Hints:* (1) "What happens if two users request the same custom alias simultaneously?" (2) "Should recording a click block the redirect response?" (3) "How would you show 'clicks per day for the last 30 days' efficiently?"
- *Follow-ups:* "How would you detect and block bot-driven click fraud inflating the analytics?" / "What if a customer wants real-time (sub-second) click counts, not just daily buckets?"
- *Rubric focus:* Correctly separating the fast redirect path from the async analytics path; DB-level uniqueness constraint reasoning.

Mock 3: Public API Rate Limiter

Candidate Brief: "Design a rate limiter that can be applied to any of our public APIs, supporting different limits per API key and per endpoint."

Interviewer Notes:

- *Expected areas:* [Ch.16, Problem 3](#)'s core, plus per-key AND per-endpoint compound limiting, and where in the request path the limiter sits (gateway-level).
- *Hints:* (1) "Token bucket or leaky bucket — what's the difference, and which fits a public API better?" (2) "What happens to legitimate traffic if your Redis cluster briefly goes down?" (3) "How would you rate-limit differently for a free tier vs. a paid tier?"
- *Follow-ups:* "How would you rate-limit at the network edge, before requests even reach your data center, to protect against a real DDoS attempt?" / "Design the response headers you'd return to help well-behaved clients self-throttle."
- *Rubric focus:* Fail-open/fail-closed trade-off explicitly stated; atomic check-and-increment reasoning.

Mock 4: Single-Channel Email Notification Service

Candidate Brief: "Design a backend service that sends transactional emails (order confirmations, password resets) triggered by other internal services."

Interviewer Notes:

- *Expected areas:* async queue-based design, templating, delivery-status tracking, retry-on-provider-failure.
- *Hints:* (1) "Should the triggering service wait for the email to actually send?" (2) "What happens if your email provider (SES/SendGrid) is rate-limited or down?" (3) "How would you know if emails are landing in spam folders at scale?"

- *Follow-ups*: "How would you extend this to also support SMS and push, sharing infrastructure? (this is Chapter 16, Problem 5 — see if the candidate derives the per-channel-queue pattern independently)" / "Design bounce and complaint handling."
- *Rubric focus*: Correctly identifying this as inherently async; DLQ/retry design.

Mock 5: Internal FAQ/Help-Center Search Tool

Candidate Brief: "Design a search tool for our internal support team to quickly find help-center articles by keyword, with typo tolerance."

Interviewer Notes:

- *Expected areas*: inverted index reasoning (even at small scale, the *concept* should be named), typo tolerance mechanism, ranking approach for a small, curated corpus.
 - *Hints*: (1) "Why wouldn't a plain SQL `LIKE '%keyword%'` query work well here?" (2) "How would you handle 'delivery' matching 'deliverty'?" (3) "Does this really need a dedicated search engine like Elasticsearch, given the corpus is probably small?"
 - *Follow-ups*: "The corpus grows to 500,000 articles across 40 languages — what changes?"
 - *Rubric focus*: Right-sizing the solution to actual scale (Elasticsearch may be overkill for a small internal tool — reward a candidate who says so, per Chapter 14.4's overengineering mistake).
-

Intermediate Tier (Mocks 6-10)

Mock 6: Instagram (Feed-Focused)

Candidate Brief: "Design the feed and upload functionality for a photo-sharing app like Instagram. Assume ~200M daily active users."

Interviewer Notes:

- *Expected areas*: capacity estimation, upload pipeline (pre-signed URL + async processing), fan-out strategy for the feed, the celebrity-account problem.
- *Hints*: (1) "How would you show a new post to 50M followers without 50M synchronous writes?" (2) "Does your fan-out approach work the same way for a regular user and a celebrity with 80M followers?" (3) "Where does the actual image data live, versus the post metadata?"
- *Follow-ups*: "How would you build the Explore tab, which isn't based on who you follow?" / "How would you handle a post being deleted after it's already been fanned out to millions of caches?"
- *Rubric focus*: Fan-out-on-write vs. fan-out-on-read reasoning, and specifically whether the candidate independently proposes a hybrid for the celebrity case rather than needing it spelled out.

Mock 7: Cinema Chain Ticket Booking

Candidate Brief: "Design a ticket booking system for a cinema chain with hundreds of screens nationwide. A new blockbuster's tickets go on sale at a specific announced time."

Interviewer Notes:

- *Expected areas:* seat-level inventory modeling, booking atomicity, the scheduled-spike problem specifically.
- *Hints:* (1) "Two users select the same seat within the same second. Walk me through exactly what happens in your system." (2) "The movie is announced to go on sale at exactly 10:00 AM. What do you do differently than for normal, steady traffic?" (3) "What happens if a user selects seats but abandons checkout?"
- *Follow-ups:* "How would you handle a network partition between the booking service and the seat-inventory database mid-transaction?" / "Design the refund flow for a cancelled screening."
- *Rubric focus:* TTL-based seat lock design; explicit admission-control/waiting-room proposal for the scheduled spike (don't accept "just autoscale" as sufficient here).

Mock 8: Dropbox

Candidate Brief: "Design a file storage and sync service. Users should be able to edit a large file on one device and have it sync efficiently to their other devices."

Interviewer Notes:

- *Expected areas:* metadata/blob split, chunking for efficient re-sync, conflict handling for offline edits.
- *Hints:* (1) "A user changes 1 line in a 2GB file. Does your system re-upload all 2GB?" (2) "Two devices edit the same file while both offline, then reconnect. What happens?" (3) "How do other devices find out a file changed, without constantly polling?"
- *Follow-ups:* "How would sharing a folder with another user change your permission model?" / "How would you support real-time collaborative editing instead of just sync?"
- *Rubric focus:* Chunking/dedup reasoning specifically — a candidate who treats a file as one atomic blob has missed the core insight of this problem.

Mock 9: Food Delivery App — Core Ordering and Tracking

Candidate Brief: "Design the core ordering and live-tracking flow for a food delivery app, from placing an order to it arriving."

Interviewer Notes:

- *Expected areas:* order state machine, restaurant accept/reject flow, delivery-partner matching, live location tracking to the customer.
- *Hints:* (1) "What happens if the restaurant doesn't respond to the order within 2 minutes?" (2) "How does the customer see the delivery partner's location update in near-real-time?" (3) "Would you store every single location ping permanently?"
- *Follow-ups:* "How would you batch multiple orders onto one delivery partner during a

lunch rush?" / "How would surge pricing work during a rain-driven demand spike?"

- *Rubric focus:* Explicit separation of the order/payment data path (strong consistency) from the location-tracking path (loss-tolerant, latest-value-wins).

Mock 10: Twitter-like Microblogging Platform

Candidate Brief: "Design a microblogging platform where users post short text updates and follow other users to see their posts in a timeline."

Interviewer Notes:

- *Expected areas:* essentially [Chapter 17, Problem 7](#) — see if the candidate derives fan-out-on-write and the celebrity exception from first principles.
 - *Hints:* (1) "A user with 50M followers posts something. Walk me through what happens, step by step." (2) "How does a regular user's timeline get built when they open the app?" (3) "Are these two cases (celebrity vs. regular user) handled the same way in your design?"
 - *Follow-ups:* "How would you add a 'retweet' feature without duplicating storage of the original post?" / "How would you rank a timeline instead of showing it purely chronologically?"
 - *Rubric focus:* Whether the hybrid fan-out resolution is reached independently or only after a strong hint — independent derivation is a senior-level signal.
-

| Senior Tier (Mocks 11-15)

Mock 11: Uber (Full Ride-Hailing Product)

Candidate Brief: "Design the core system for a ride-hailing app: riders request rides, drivers accept and complete them, and pricing adjusts to demand."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problems 17 & 23](#) — geospatial matching, trip state machine, surge pricing.
- *Hints:* (1) "How do you find the nearest available driver among a million active drivers, in under a second?" (2) "What data does your surge-pricing calculation actually need, and how fresh does it need to be?" (3) "Two riders get matched to the same driver simultaneously — how do you prevent that?"
- *Follow-ups (push hard here — this is a senior-tier mock):* "Your matching service is returning results in 3 seconds instead of the required 300ms during peak hours. Walk me through how you'd diagnose and fix this." / "How would you handle a driver's phone losing signal mid-trip?"
- *Rubric focus:* Atomic assignment correctness under the double-match challenge; whether the candidate proactively estimates capacity for the location-ping pipeline without being asked.

Mock 12: Payment Gateway

Candidate Brief: "Design a payment gateway that merchants integrate with to accept card payments, similar to Stripe or Razorpay."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 21](#) and [Chapter 20](#) in full — idempotency, state machine, reconciliation, webhooks.
- *Hints:* (1) "A client retries a payment request after a timeout. How do you guarantee they're not charged twice?" (2) "Your gateway calls the card network, but the response is lost before you receive it. What state is the transaction in, and how do you resolve it?" (3) "How does the merchant find out a payment succeeded if it wasn't synchronous?"
- *Follow-ups (this should feel like a real, hard interview):* "Design the reconciliation job specifically — what does it compare, and how does it resolve discrepancies?" / "A merchant claims they never received a webhook for a successful payment. Walk me through how you'd investigate and what would prevent this going forward."
- *Rubric focus:* This mock should be scored primarily on the idempotency and unknown-outcome-handling depth — a candidate who doesn't independently raise "what if the response is lost" has missed the central challenge of this domain.

Mock 13: WhatsApp at Scale

Candidate Brief: "Design a messaging system supporting over a billion users, including 1:1 chats, group chats up to 256 members, and message delivery even when a recipient is offline."

Interviewer Notes:

- *Expected areas:* [Chapter 17, Problem 9](#) — connection routing, durable-first delivery, offline push-notification handoff.
- *Hints:* (1) "Sender and recipient are connected to different connection servers. How does a message find its way from one to the other?" (2) "The recipient's phone is off. What happens to the message, and how do they eventually get it?" (3) "How is message ordering guaranteed within one conversation?"
- *Follow-ups:* "How would end-to-end encryption change what your backend can and can't do with message content?" / "A group has 256 members, all online simultaneously. Walk through sending one message to all of them."
- *Rubric focus:* Presence/routing-layer design correctness; explicit statement of the durability-vs-live-delivery split.

Mock 14: E-commerce Flash-Sale System

Candidate Brief: "Your e-commerce platform is running a flash sale: 10,000 units of a popular product, discounted 70%, available for exactly 1 hour, expected to draw 2 million interested shoppers."

Interviewer Notes:

- *Expected areas:* [Chapter 17, Problem 13](#) and [Chapter 18, Problem 19](#)'s flash-sale-specific content — capacity pre-provisioning, atomic inventory decrement, possibly a waiting room.
- *Hints:* (1) "2 million shoppers, 10,000 units — what's your plan for the moment the sale opens?" (2) "How do you guarantee exactly 10,000 units sell, no more, under massive concurrency?" (3) "Should everyone hit your checkout service at the exact same millisecond?"
- *Follow-ups:* "The sale is 3 days away and known in advance. What would you do differently than for organic, unpredictable traffic growth?" / "How would you prevent bots from buying all 10,000 units in the first second?"
- *Rubric focus:* Proactive mention of pre-event capacity planning (not just reactive autoscaling) and a considered admission-control strategy — both expected unprompted at this tier.

Mock 15: Multi-Channel Notification System for Millions of Users

Candidate Brief: "Design a notification system that can send a single campaign to 50 million users across push, SMS, and email, while also supporting urgent, low-volume transactional alerts like OTPs that must be delivered within seconds."

Interviewer Notes:

- *Expected areas:* [Chapter 16, Problem 5](#) and [Chapter 18, Problem 28](#) combined — per-channel queues, and critically, priority-lane separation between bulk and transactional traffic.
- *Hints:* (1) "Your 50M-user campaign is running. An OTP needs to go out right now for an unrelated login. Does it wait behind the campaign?" (2) "Your SMS provider caps at 200 requests/second. Do the math — how long does 50M actually take, realistically?" (3) "How would you know, mid-campaign, if delivery is falling behind?"
- *Follow-ups:* "Design the fallback cascade for a security alert that fails to deliver via push within 30 seconds."
- *Rubric focus:* This mock should specifically fail a candidate who proposes one uniform pipeline for both bulk and transactional traffic without prompting — separating them is the entire point.

Tier-1 (Mocks 16-20)

Mock 16: Amazon — Full Marketplace and Fulfillment

Candidate Brief: "Design Amazon's core marketplace: customers browse and buy from millions of products listed by both Amazon and third-party sellers, fulfilled from a network of regional warehouses."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 19](#) — multi-tenant catalog, Buy Box ranking, warehouse-fulfillment routing, plus everything from the base e-commerce problem.
- *Hints (offer sparingly — this tier should feel genuinely hard):* (1) "The same product is sold by 12 different sellers. What does the customer actually see, and how do you decide?" (2) "An order needs to ship. Which of your dozens of warehouses fulfills it, and why?"
- *Follow-ups (expect and push for these, Amazon-interview-style):* "How does this design handle Prime Day, at 10x your normal peak multiplier?" / "Walk me through the trade-offs of your Buy-Box ranking approach — what could go wrong, and how would you know?" / "How would you justify the cost of your proposed architecture to a VP who's asking why it's not simpler?"
- *Rubric focus:* Cost-awareness and business-impact reasoning are explicitly in scope at this tier, not just technical correctness (Chapter 14.3's staff-level distinction) — score down an answer that's technically sound but never once mentions cost, ownership, or business trade-offs.

Mock 17: Netflix — Personalization and Global Streaming

Candidate Brief: "Design Netflix's home page and video delivery: a highly personalized per-user home page, and video streaming that adapts to each viewer's network conditions, for 250M+ subscribers across licensing-restricted regions."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 20](#) — precomputed personalization, adaptive bitrate streaming, regional licensing enforcement.
- *Hints:* (1) "200M users open the app at 8 PM their local time. Are you computing their personalized home page live, right then?" (2) "A title's licensing expires in your catalog for one specific region tomorrow. How does your system enforce that automatically?"
- *Follow-ups:* "How would you A/B test a new ranking algorithm safely, at this scale, without risking a broad regression in engagement?" / "A regional ISP is throttling your CDN traffic. What are your options?"
- *Rubric focus:* Explicit "precompute expensive, serve fast" reasoning; whether licensing is treated as a first-class architectural constraint (in the data model) or an afterthought.

Mock 18: Digital Wallet with Reconciliation (PhonePe-style)

Candidate Brief: "Design a digital wallet product: users hold a balance, transfer money to each other, top up from a bank account, and your system must reconcile against bank settlement files daily. This is a regulated financial product."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 22](#) and [Chapter 20](#) in full depth — this mock should closely mirror PhonePe's actual reported interview content (Chapter 19).
- *Hints:* (1) "Explain exactly how you'd represent a user's balance in your data model —

walk me through a P2P transfer at the data level." (2) "Your settlement file from the bank shows a transaction your system has no record of. What happened, and how do you resolve it?"

- *Follow-ups (go deep, this should be a genuinely hard 60 minutes):* "A regulator asks you to prove, for any given day, that your total wallet liabilities exactly match your actual bank reserves. How does your design make that provable?" / "How would you detect and prevent an engineer's bug from silently creating money that doesn't exist?"

- *Rubric focus:* This is the single highest-bar mock in the roadmap for your stated goals — double-entry correctness and the debits-equal-credits invariant should be produced unprompted, not extracted via hints.

Mock 19: Careem — Super-App Architecture

Candidate Brief: "Design the platform architecture for a super-app offering ride-hailing, food delivery, and an in-app wallet, all under one account, with shared payment and identity across all three."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 18](#) — the vertical-vs-shared-platform-service boundary question specifically.

- *Hints:* (1) "Should ride-matching and food-delivery-matching share the same underlying dispatch infrastructure, or be entirely separate?" (2) "A user pays for a ride using their in-app wallet balance. Which service owns that operation?"

- *Follow-ups:* "A new vertical — grocery delivery — is being added next quarter. Which of your existing services does it reuse, and which does it need to build fresh?" / "How do you prevent a bug in the food-delivery service from being able to directly corrupt wallet balances?"

- *Rubric focus:* Whether the candidate proposes a principled boundary (shared platform capabilities vs. independent vertical logic) rather than either full duplication or a single monolithic shared service — this is the specific test this mock exists for.

Mock 20: Global Real-Time Ad Auction System

Candidate Brief: "Design a real-time ad-serving system: given an ad opportunity (a page view), select and return the winning ad from thousands of eligible campaigns within a strict end-to-end latency budget, including calls to external bidders."

Interviewer Notes:

- *Expected areas:* [Chapter 18, Problem 27](#) — eligibility filtering, auction/ranking, budget pacing, hard external-bidder timeouts.

- *Hints:* (1) "You have a 100ms total budget and need to call 5 external bidders. What happens if one of them is slow?" (2) "A campaign has a \$10,000 daily budget. You're serving from 200 servers globally. How do you prevent overspending without every single ad request hitting one central budget counter?"

- *Follow-ups (Google/Meta-tier depth expected):* "Explain second-price auction mechanics

and why that design choice matters for bidder behavior." / "How would you detect and prevent a bug that's serving one advertiser's ads far more than their bid/budget would justify?"

- *Rubric focus:* The budget-pacing distributed-coordination trade-off is the crux of this mock — a candidate proposing a single global synchronous counter checked on every request has missed the core latency constraint of the problem.

Next → [Chapter 29: Evaluation Rubric & Final Framework](#) — score yourself out of 100 on any mock above, and the one-page decision tree for designing any unfamiliar product from first principles.